

Expense Splitter Application

Overview

In this project, you will work in **pairs** to design and implement a **backend application** for splitting expenses between multiple users (similar to Splitwise).

The goal is to practice:

- Spring Boot development
 - Security and authentication
 - Clean architecture
 - Exception handling
 - Unit testing
 - Team collaboration using Git & GitHub
-

Objectives

By completing this assignment, you should be able to:

- Build a RESTful API with Spring Boot
 - Implement authentication and authorization using Spring Security
 - Design relational data models
 - Apply business logic for financial calculations
 - Write unit tests using JUnit
 - Collaborate effectively using Git
-

Functional Requirements



User Management

Features:

- User registration
- User login

- Password encryption (BCrypt)

Fields:

- id
 - username
 - email
 - password
 - role (USER, ADMIN)
-

** Security (Spring Security)****Requirements:**

- Authentication (login)
- Authorization based on roles
- Only authenticated users can access the API

Authorization rules:

- USER:
 - Can manage their own expenses
 - ADMIN:
 - Can view all users and all expenses
-

Expense Management**Create Expense**

A user can create a new expense with:

- description
- total amount
- participants (list of users)
- paidBy (user who paid)

Business Logic (CORE REQUIREMENT)

The application must:

- Split the expense **equally** among participants
- Calculate how much each participant **owes**

Example:

- Total: 30\$
- Participants: A, B, C (3 users)
- Paid by: A

 Result:

- Each owes: 10
- A paid full amount:
 - B owes A → 10
 - C owes A → 10

Expense Summary

Users should be able to:

- View all their expenses
- View balances:
 - Who they owe money to
 - Who owes them money

Data Model (Suggested)

Entities:

- User
- Expense

- ExpenseParticipant (join table)
-

Testing (JUnit)

You must write unit tests for:

- Expense creation logic
- Expense splitting calculation
- Authorization checks

Minimum:

- 3–5 meaningful test cases
-

API Requirements (Suggested)

You should implement REST endpoints such as:

User:

POST /api/auth/register

POST /api/auth/login

GET /api/users

Expenses:

POST /api/expenses

GET /api/expenses

GET /api/expenses/{id}

Balance:

GET /api/balance

Project Structure (Suggested)

controller/

service/

repository/

model/

dto/

exception/

security/

Bonus Features (Optional)

- Unequal split (advanced)
 - Settlement feature ("mark as paid")
 - Pagination & sorting
 - Docker setup
 - JWT authentication instead of session-based
-

Timeline (3 Weeks)

Week 1:

- Project setup
- User entity and authentication
- Basic CRUD

Week 2:

- Expense logic
- Security setup
- Role-based access

Week 3:

- Exception handling
- Testing
- Final improvements

Deliverables

You must submit:

1. ☒ GitHub repository
2. ☒ README file including:
 - Project description
 - Setup instructions
 - API description
3. ☒ Working application
4. ☒ Unit tests

Notes

- Focus on **clean code and correctness**, not UI
- You may test APIs using Postman
- Collaboration and version control are part of the grade